

Universidad Nacional Abierta y a Distancia

Vicerrectoría Académica y de Investigación

Unidad gestora: *Escuela de Ciencias Básicas Tecnología e Ingeniería ECBTI*

Programa: *Ingeniería de Sistemas*

Curso: Programación

Código: 213023

Ejercicio 1: Sistema Integral de Gestión de Clientes, Servicios y Reservas

En un equipo conformado por cinco (5) estudiantes, deberán desarrollar un **sistema integral orientado a objetos**, sin uso de bases de datos, capaz de gestionar clientes, servicios y reservas para una empresa llamada Software FJ que ofrece varios tipos de servicios (reservas de salas, alquiler de equipos y asesorías especializadas). El objetivo de esta tarea es construir una aplicación **estable, modular y extensible** que implemente de forma rigurosa los principios de **abstracción, herencia, polimorfismo, encapsulación y manejo avanzado de excepciones**, garantizando que el sistema siga funcionando aun cuando se presenten errores durante su ejecución.

El sistema debe incluir clases abstractas, clases derivadas, métodos sobrecargados, manejo de listas internas y validaciones estrictas, demostrando un diseño orientado a objetos completamente funcional. La información no debe almacenarse en bases de datos; toda la gestión debe hacerse mediante objetos, listas y manejo de archivos únicamente para el registro de eventos y errores.

Como parte esencial de la tarea, el sistema deberá incorporar **manejo robusto de excepciones**, incluyendo excepciones personalizadas, uso de bloques try/except, try/except/else, try/except/finally, y encadenamiento de excepciones. Cada error detectado debe registrarse en un archivo de logs, manteniendo la aplicación activa y estable en todo momento. El sistema debe ser capaz de manejar errores provenientes de datos inválidos, parámetros faltantes, operaciones no permitidas, intentos de reserva incorrectos, servicios no disponibles, cálculos inconsistentes y cualquier otra situación que pueda comprometer la operación normal de la aplicación.

El trabajo debe basarse en una arquitectura orientada a objetos, incluyendo al menos:

- Una clase abstracta que represente entidades generales del sistema.
- Una clase Cliente con validaciones robustas y encapsulación de datos personales.
- Una clase abstracta Servicio y al menos tres servicios especializados que hereden de ella, implementando polimorfismo y métodos sobrescritos para calcular costos, describir servicios y validar parámetros.
- Una clase Reserva que integre cliente, servicio, duración y estado, e implemente confirmación, cancelación y procesamiento con manejo de excepciones.
- Métodos sobrecargados (por ejemplo, diferentes variantes del cálculo de costos con impuestos, descuentos o parámetros opcionales).

- Un archivo de logs donde se registren todos los errores y eventos relevantes.

El sistema, sin utilizar ningún motor de base de datos, debe simular al menos **10 operaciones completas**, incluyendo registros válidos e inválidos de clientes, creación correcta e incorrecta de servicios, y reservas exitosas y fallidas, demostrando la capacidad del programa para continuar funcionando ante errores graves y manejar excepciones de manera controlada y profesional.

El equipo debe entregar un único proyecto completamente funcional, organizado, documentado y capaz de ejecutarse sin interrupciones, demostrando la correcta aplicación de la programación orientada a objetos y el manejo avanzado de excepciones en un entorno sin base de datos.

Desarrollo de la actividad

Introducción

Este proyecto consiste en el desarrollo de un sistema integral orientado a objetos para la empresa Software FJ, capaz de gestionar clientes, servicios y reservas sin utilizar bases de datos. El sistema fue desarrollado en Python aplicando los principios de Programación Orientada a Objetos como abstracción, herencia, encapsulación y polimorfismo, además del manejo avanzado de excepciones y registros de eventos mediante archivos de logs.

Objetivos

Desarrollar un sistema orientado a objetos en Python para la gestión de clientes, servicios y reservas de la empresa Software FJ, aplicando principios de programación orientada a objetos y manejo de excepciones.

Objetivos específicos

Implementar clases abstractas y clases derivadas.

Aplicar encapsulación, herencia y polimorfismo.

Gestionar reservas y servicios mediante listas.

Implementar manejo de excepciones personalizadas.

Registrar eventos y errores en archivos de logs.

Explicación del sistema

cliente.py:

Contiene la clase Cliente y las validaciones de datos personales.

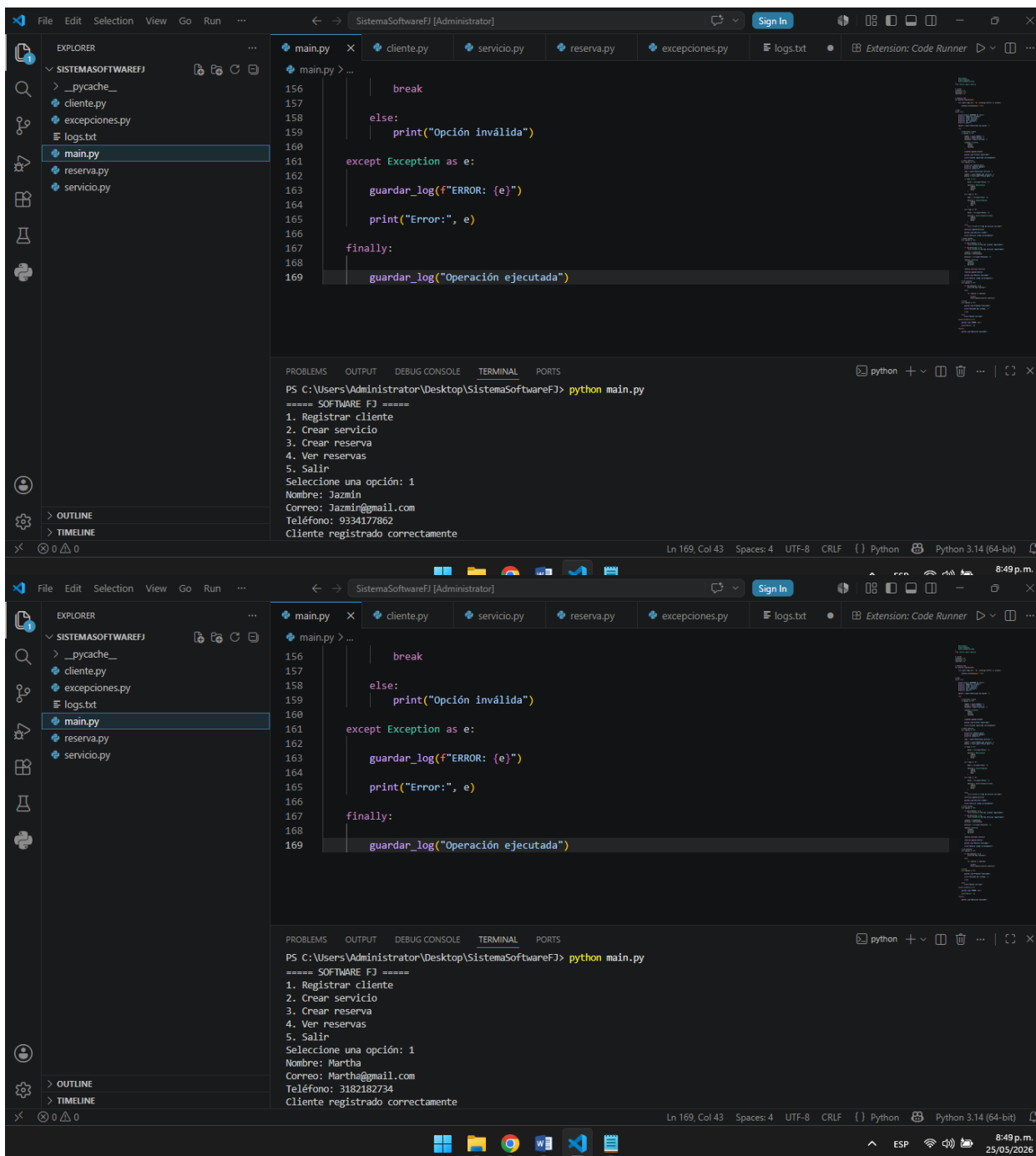
servicio.py:

Contiene la clase abstracta Servicio y los servicios derivados.

reserva.py:

Gestiona las reservas y el estado de las mismas.

CAPTURAS DE PYTHON



The image shows two screenshots of the Visual Studio Code editor. The top screenshot shows the execution of a Python program named `main.py` in the `SistemaSoftwareFJ` project. The program is running in a terminal window, and the output shows the following steps:

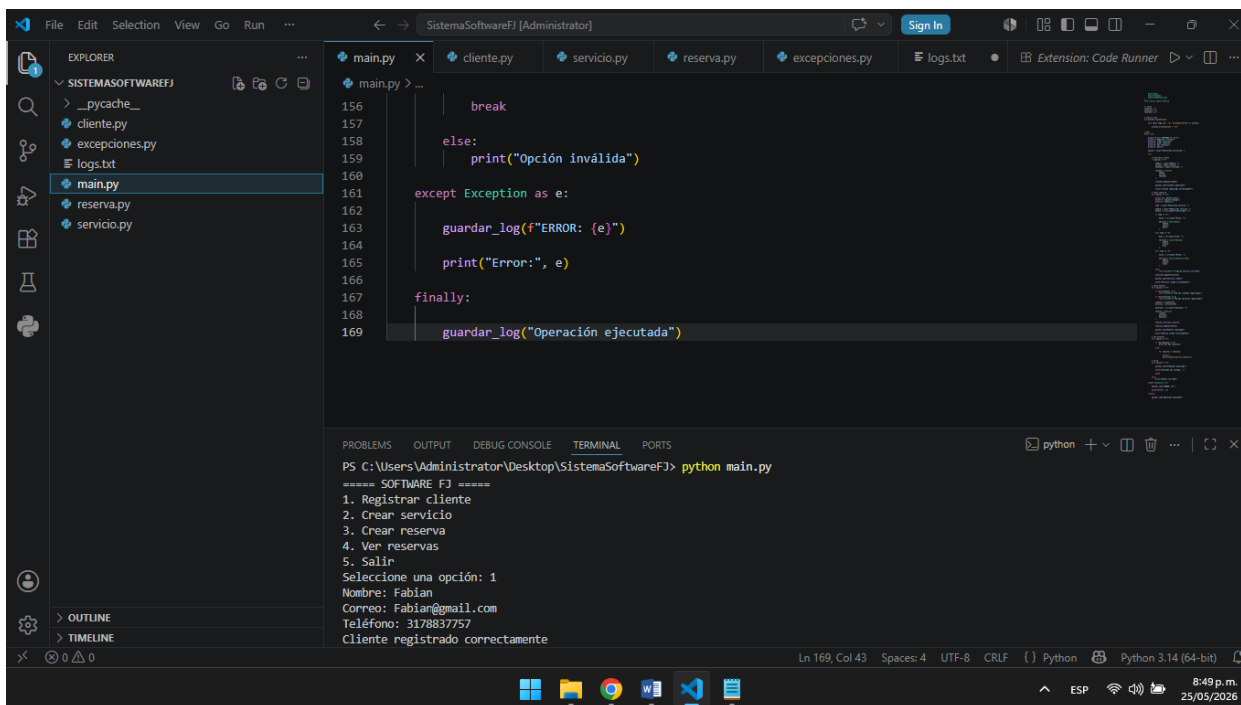
```
PS C:\Users\Administrator\Desktop\SistemaSoftwareFJ> python main.py
===== SOFTWARE FJ =====
1. Registrar cliente
2. Crear servicio
3. Crear reserva
4. Ver reservas
5. Salir
Seleccione una opción: 1
Nombre: Jazmin
Correo: Jazmin@gmail.com
Teléfono: 9334177862
Cliente registrado correctamente
```

The bottom screenshot shows the same program running, but with a different input for the name: `Martha`. The output is:

```
PS C:\Users\Administrator\Desktop\SistemaSoftwareFJ> python main.py
===== SOFTWARE FJ =====
1. Registrar cliente
2. Crear servicio
3. Crear reserva
4. Ver reservas
5. Salir
Seleccione una opción: 1
Nombre: Martha
Correo: Martha@gmail.com
Teléfono: 3182182734
Cliente registrado correctamente
```

The code in `main.py` is as follows:

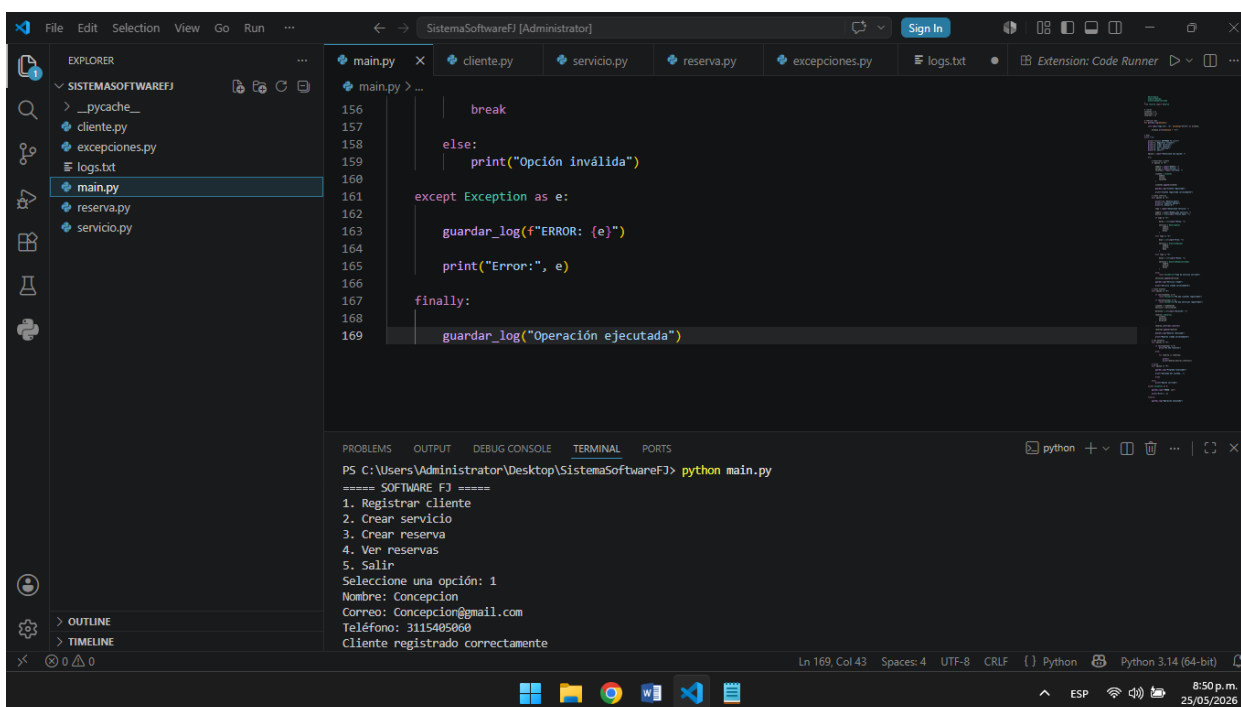
```
156         break
157     else:
158         print("Opción inválida")
159
160
161 except Exception as e:
162
163     guardar_log(f"ERROR: {e}")
164
165     print("Error:", e)
166
167 finally:
168
169     guardar_log("Operación ejecutada")
```



```

File Edit Selection View Go Run ...
SistemaSoftwareFJ [Administrator]
main.py cliente.py servicio.py reserva.py excepciones.py logs.txt Extension: Code Runner
EXPLORER
SISTEMASOFTWAREFJ
  _pycache_
  cliente.py
  excepciones.py
  logs.txt
  main.py
  reserva.py
  servicio.py
main.py
156
157
158
159
160
161
162
163
164
165
166
167
168
169
break
else:
    print("Opción inválida")
except Exception as e:
    guardar_log(f"ERROR: {e}")
    print("Error:", e)
finally:
    guardar_log("Operación ejecutada")
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Administrator\Desktop\SistemaSoftwareFJ> python main.py
===== SOFTWARE FJ =====
1. Registrar cliente
2. Crear servicio
3. Crear reserva
4. Ver reservas
5. Salir
Seleccione una opción: 1
Nombre: Fabian
Correo: Fabian@gmail.com
Teléfono: 3178837757
Cliente registrado correctamente
Ln 169, Col 43 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit)
8:49 p.m. 25/05/2026

```



```

File Edit Selection View Go Run ...
SistemaSoftwareFJ [Administrator]
main.py cliente.py servicio.py reserva.py excepciones.py logs.txt Extension: Code Runner
EXPLORER
SISTEMASOFTWAREFJ
  _pycache_
  cliente.py
  excepciones.py
  logs.txt
  main.py
  reserva.py
  servicio.py
main.py
156
157
158
159
160
161
162
163
164
165
166
167
168
169
break
else:
    print("Opción inválida")
except Exception as e:
    guardar_log(f"ERROR: {e}")
    print("Error:", e)
finally:
    guardar_log("Operación ejecutada")
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Administrator\Desktop\SistemaSoftwareFJ> python main.py
===== SOFTWARE FJ =====
1. Registrar cliente
2. Crear servicio
3. Crear reserva
4. Ver reservas
5. Salir
Seleccione una opción: 1
Nombre: Concepcion
Correo: Concepcion@gmail.com
Teléfono: 3115405000
Cliente registrado correctamente
Ln 169, Col 43 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit)
8:50 p.m. 25/05/2026

```

Conclusiones

Durante el desarrollo del proyecto se aplicaron los conceptos fundamentales de Programación Orientada a Objetos, logrando construir un sistema funcional y estable. Además, se implementó manejo de

excepciones y registros de eventos mediante logs, permitiendo que el sistema continúe funcionando aun cuando se presentan errores.

Referencias bibliográficas

- Python Software Foundation. (2025). Python Documentation.
- Sweigart, A. (2019). Automate the Boring Stuff with Python.
- Deitel, P., & Deitel, H. (2019). Python for Programmers.
- Lutz, M. (2013). Learning Python (5th ed.).
- Oracle. (2025). Object-Oriented Programming Concepts.

Nombre Estudiante: Fabián Stiven Ballen Neira

Tutora: Martha Liliana Quevedo